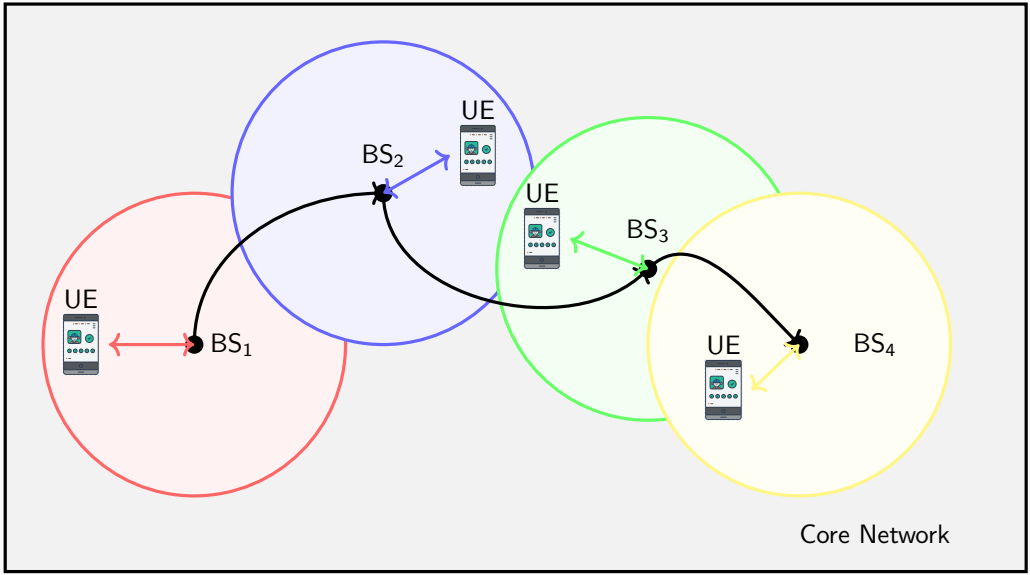
Formalizing Secure Handovers Schemes

IEEE CSF 2025 Accepted Paper Presentation

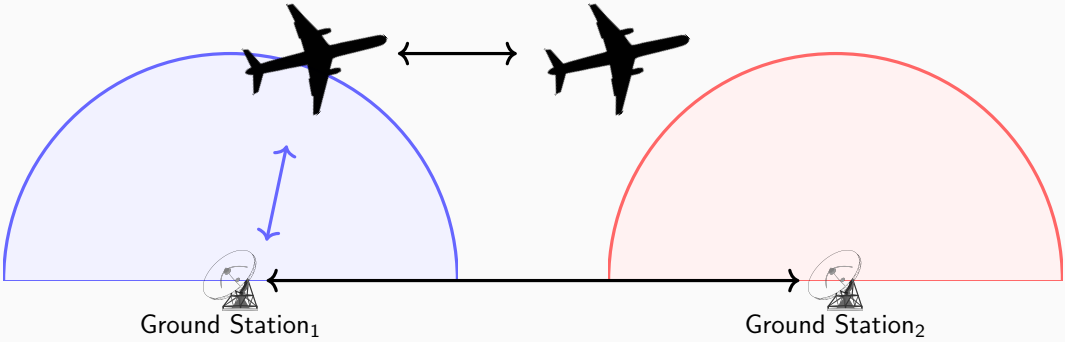
Rabiah Alnashwan RAlnashwan1@sheffield.ac.uk
Benjamin Dowling benjamin.dowling@kcl.ac.uk
Bhagya Wimalasiri b.m.wimalasiri@sheffield.ac.uk

2025

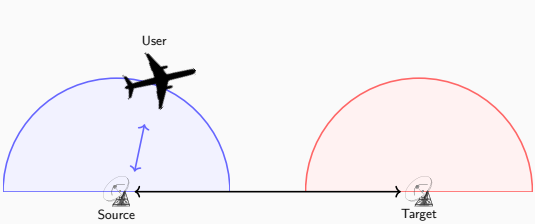
5G Handover : The Canonical Example

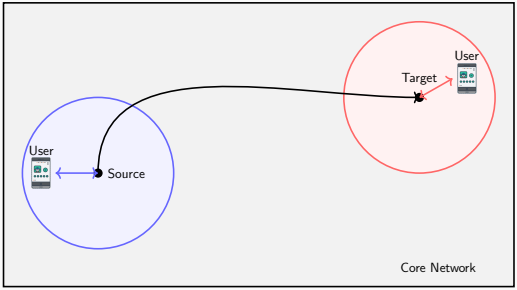


LDACS Handover



Common Features of Handovers





Consistently at least three parties: User `U`, Source `S` and Target `T`.

- Source and User have pre-established some relationship.
- Source and Target have some pre-defined relationship.
- User wants to "continue session" at Target.

Common Features II

Users are typically mobile

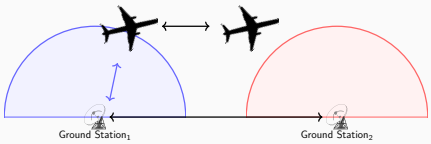
- Handovers broadly maps a User's path through a series of nodes in a Network

Hardware limitations:

- Users are phones, planes, satellites, ...
- Sometimes quite strapped for storage or computation!

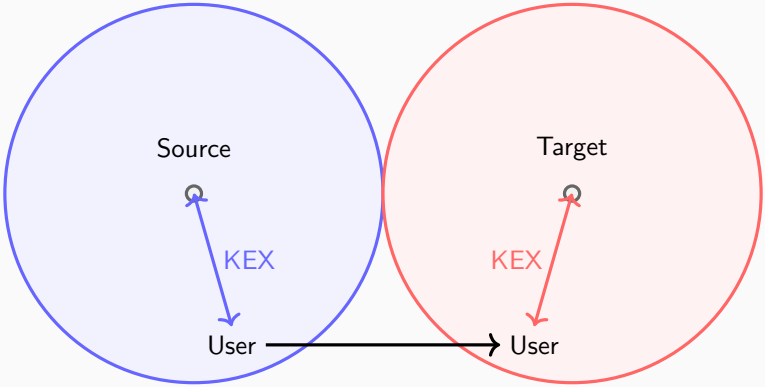
Network Limitations:

- Communication mediums themselves may differ based on the party (LDACS).
- All parties may not communicate synchronously(LDACS).



Session Handover vs Key Exchange

Previous literature mostly modelled handover protocols as **key exchange**.



This does achieve the wanted functionality... however!

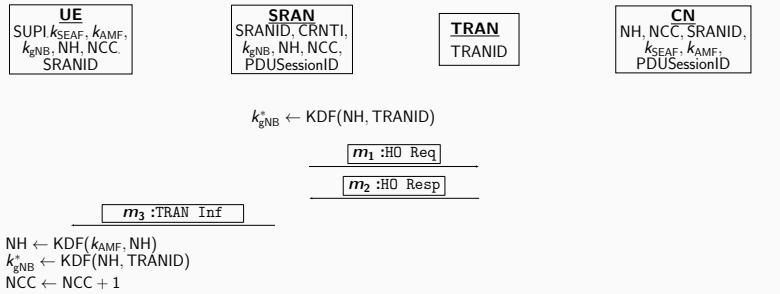
SHO vs KEX: Keys

KEX (broadly) assumes all parties have **similar** secrets

- i.e. all preshared keys or all asymmetric keys.

In Handovers, Users are typically different from **Nodes**:

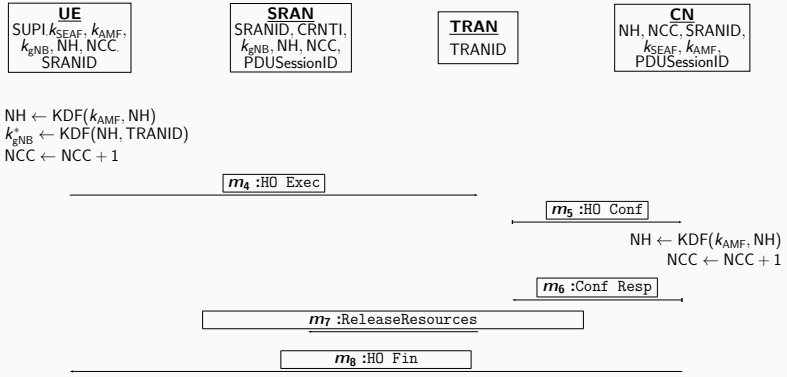
- 5G: UE shares a *symmetric key* with CN, but SN/TN authenticate using *asymmetric keys*
- LDACS: Source shares a *session key* with User, but Target authenticates via *digital signatures*.



SHO vs KEX: Session Continuity

Treating SHO as KEX means each new node is "fresh" session. No ephemeral state shared between subsequent KEX sessions.

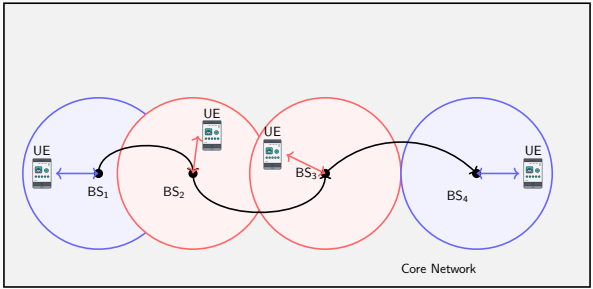
- In practice, SHO is a **continuance** of existing session.
- 5G Handover: CN syncs UE values consistently between Target and Source.



SHO vs KEX: Who Keeps the Keys?

Treating SHO as 3P-KEX implies all parties share keys after protocol execution.

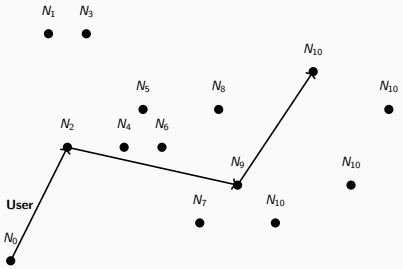
- In SHO, only User and Target should share the new session key.
- User and Target may want to ensure Source cannot know this key!



SHO vs KEX: Path Privacy?

Treating SHO as independent KEX obscures **path knowledge**.

- Handovers can leak geographic information about User.
- User may not want Source and Target to know path.

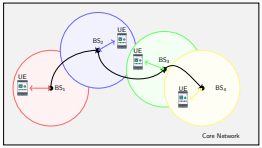


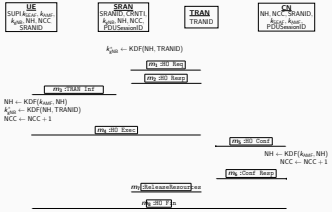
Even knowing identities of **Source** and **Target** may leak private geographic information.

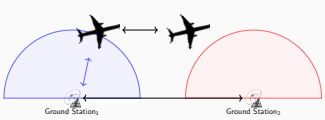
SHO vs KEX: Summary

SHO it's own thing, identified commonalities in these protocols:

- **SHO** assumes relationship between User and Source: abstract "bootstrap" secret.
- Source can authenticate on behalf of User to Target.
- Handovers can be done in sequence: **SHO** should generate future "handover keys".
- Source should **not** learn handover keys, despite access to bootstrap key.
- To prevent *tracking attacks*, want **path privacy guarantees**.



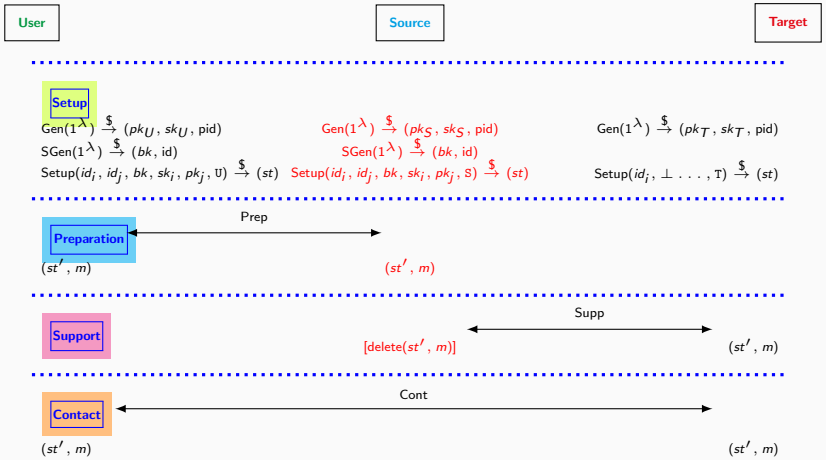




How to formalize these notions?

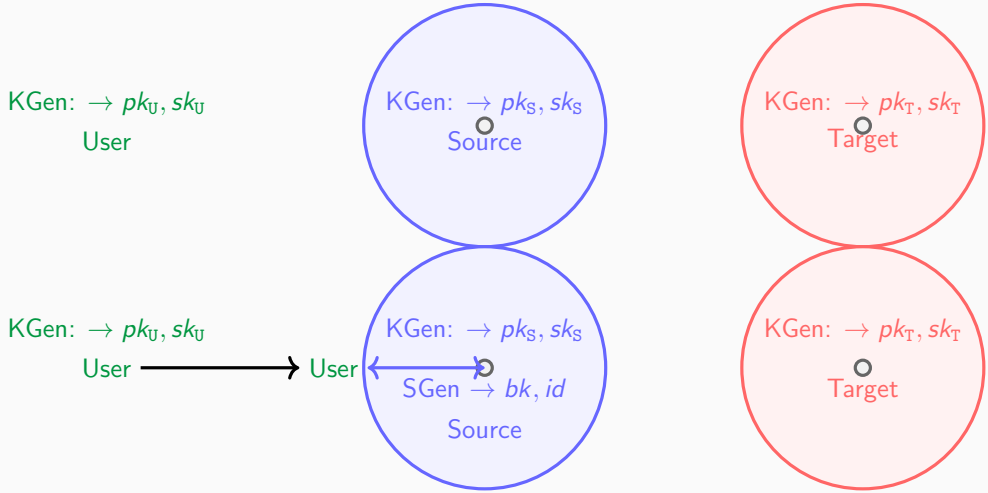
Formalizing Secure Handovers

Formalizing Secure Handovers



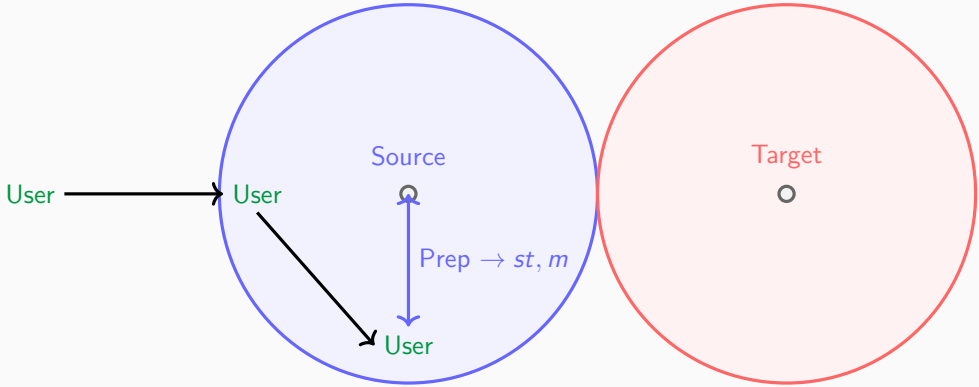
- 3 Parties: User, Source, Target.
- 4 Phases: **Setup**, **Prep**, **Support**, **Contact**.
- 6 Algorithms: **KGen**, **SGen**, **Setup**, **Prep**, **Support**, **Contact**.

Formalizing Secure Handovers: Setup Phase



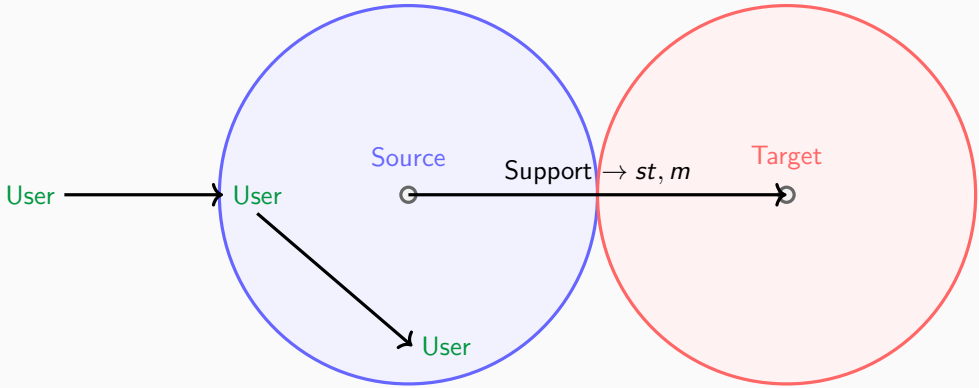
- **KGen** generates any long-term keys, secrets.
- **SGen** establishes bootstrap secrets.
- **Setup** does session setup.

Formalizing Secure Handovers: Preparation Phase



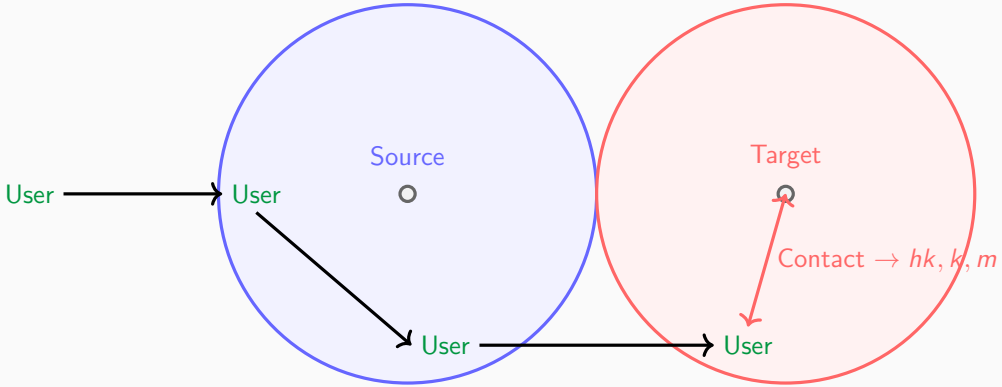
- User and Source communicate, User indicates they are doing handover.
- Enables **Path Privacy** by allowing User to act as Source proxy.
- If Source and Target don't communicate, Source provides tokens to User.

Formalizing Secure Handovers: Support Phase



- Source and Target communicate, reducing communication complexity for **User** if necessary.
  - This is done in LDACS
- Source deletes all handover state once complete.

Formalizing Secure Handovers: Contact Phase



- User and Target communicate.
- Allows User and Target to continue old session from Source.
- Establishes session and handover keys.

Formalizing Security of Handover Protocols

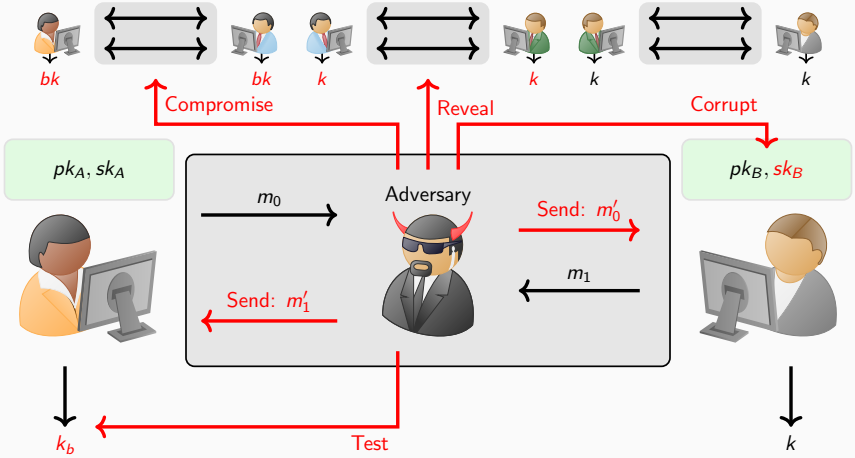
Bespoke SHO security model: What do we need?

Broadly speaking, need to define four parts to our model.

1. Adversary Goals
   - What does it mean to break this protocol?
2. Oracle queries
   - Captures the adversary's abilities.
   - Corrupt long-term keys; Reveal message keys, etc.
3. Freshness Condition
   - May want the adversary to compromise "independent" sessions, but not "challenge" sessions.
   - Need some way to define the "max" amount of corruptions to a challenge session the adversary is able to do.
   - Freshness Condition codifies trivial wins.
4. Matching Sessions
   - Secure Messaging means that sessions are *shared*
   - Define trivial win: target partner session, corrupt, decrypt "challenge ciphertext"
   - Matching Sessions defines which sessions "match"

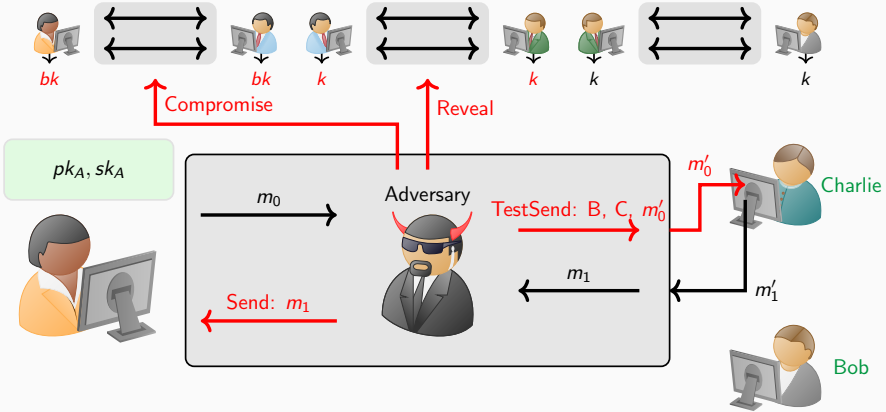
Security Game: Key Indistinguishability

Standard notions of Key Indistinguishability



Security Game: Unlinkability

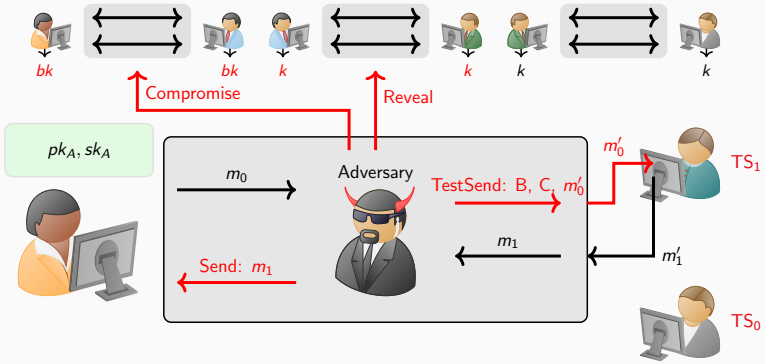
Unlinkability: Adversary tries to determine which party a "challenge" session is linked to.



These are *just* User sessions, capturing anonymity against **outsider attacks**, i.e. $\mathcal{A}$ cannot corrupt matching Source or Target session.

Security Game: Path Privacy

SPRIV + TPRIV $\implies$ **PPRIV**: $\mathcal{A}$ determines which node a "challenge" session is linked to.

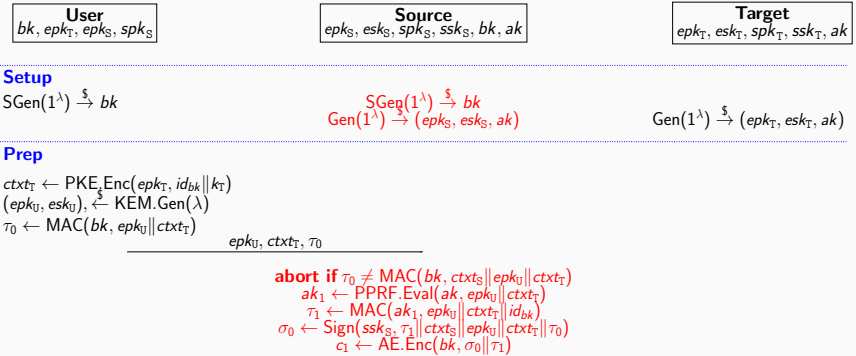


These are just variants of the **Unlinkability** game for Source / Target sessions. However, adversary is allowed to fully corrupt the *other* Target / Source, capturing **insider attacks**.

Strong Handover Protocol

Strong Handover - Setup + Prep

- **Core idea**: U uses bk+**MAC** to authenticate to S.
- U encrypts T secret+session info under pk_T, S MACs $ctxt_T, epk_U$ keyed with **PPRF** for T.
- S signs for U, U verifies and strips σ_S.



Strong Handover: Contact

- T uses **PPRF** keys to authenticate epk_U, $ctxt_T$ from S.
- T generates **KEM** ciphertext to create **PFS** session key.
- T authenticates using **MAC** keyed from secret in $ctxt_T$.

User	**Source**	**Target**
bk, epk_T, epk_S, spk_S	$epk_S, esk_S, spk_S, ssk_S, bk, ak$	$epk_T, esk_T, spk_T, ssk_T, ak$

Contact

$epk_U, ctxt_T, \tau_1$ →

$(id_{bk} \| k_T) \leftarrow \mathsf{PKE.Dec}(esk_T, ctxt_T)$
$ak_1 \leftarrow \mathsf{PPRF}(ak, epk_U \| ctxt_T)$
abort if $\tau_1 \neq \mathsf{MAC}(ak_1, epk_U \| ctxt_T \| id_{bk})$
$(ctxt_U, k_U) \xleftarrow{\$} \mathsf{KEM.Encaps}(epk_U)$
$(k, hk, mk') \leftarrow \mathsf{KDF}(k_U, k_T)$
$\tau_2 \leftarrow \mathsf{MAC}(mk', id_{bk} \| ctxt_U \| epk_U \| ctxt_T \| \tau_1)$

← $ctxt_U, \tau_2$

$k_U \xleftarrow{\$} \mathsf{KEM.Decaps}(esk_U, ctxt_U)$
$(k, hk, mk') \leftarrow \mathsf{KDF}(k_U, ck)$
abort if $\tau_2 \neq \mathsf{MAC}(mk', id_{bk} \| ctxt_U)$

Summary

- Identified a gap in how real-world handover schemes are formalized.
- Introduced our own syntax: **Secure Handover Schemes**.
- Identified new security goal for **SHO** schemes: **Path Privacy**
- Introduced and proved our **Strong Handover Protocol**.
- Future work: Model other SHOs within our framework: OAuth?

